

PreczBezrobocie

Spis treści

Wymagania użytkownika	2
Diagram przypadków użycia	3
Wymagania niefunkcjonalne systemu	3
Diagramy klas, analityczny i projektowy	4
Scenariusz przypadków użycia dla przypadku Dodawanie nowej aplikacji kandydata	5
Diagram stanu dla klasy JobOffer	6
Propozycje GUI	7
Omówienie decyzji projektowych i skutków analizy dynamicznej.	8
Technologia i architektura projektu	8
Skutki analizy dynamicznej i decyzje projektowe	8
Potencjalne rozwinięcia systemu	10

Wymagania użytkownika

System „PreczBezrobocie” służy do obsługi ofert pracy publikowanych przez firmy współpracujące z agencją pracy o nazwie „PreczBezrobocie”. Aplikacja wspiera proces rekrutacyjny prowadzony przez pracowników agencji oraz umożliwia zarządzanie ogłoszeniami, kandydatami i ich aplikacjami.

Do systemu dostęp posiada wyłącznie pracownik firmy oraz system (aktor czasowy). Aplikacja nie posiada logowania, ponieważ komputer posiada odpowiednie zabezpieczenia przed osobami nieupoważnionymi.

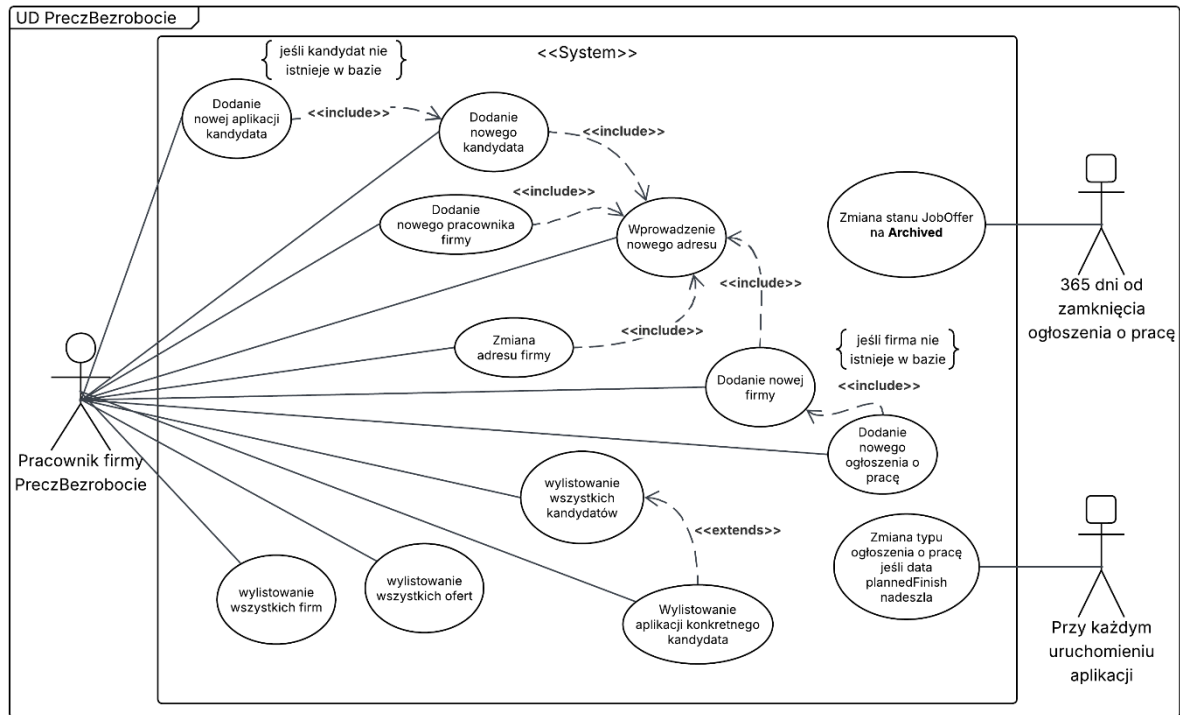
- Pracownik firmy przyjmuje zgłoszenia od firm chcących opublikować ogłoszenie o pracę.
- Pracownik może wylistować wszystkie firmy.
- Pracownik może wylistować wszystkich kandydatów oraz ich zgłoszenia.
- Jeśli dana firma nie istnieje jeszcze w bazie danych, pracownik tworzy jej rekord w systemie.
- Każdy podmiot zgłaszający się do „PreczBezrobocie” – zarówno firma, jak i osoba fizyczna – musi podać przypisany adres. Adres jest przechowywany jako osobna encja i powiązany z odpowiednim rekordem za pomocą klucza obcego.
- Firmy mają możliwość zmiany adresu. Informacja o nowym adresie jest aktualizowana przez pracownika w systemie. Pracownik dodaje również nowy adres dla firmy. Poprzedni adres firmy zostaje zachowany w historii adresów (**AddressHistory**)
- Firma może dodawać nowe ogłoszenia o pracę które do systemu wprowadza pracownik, jeśli firma nie istnieje, należy ją najpierw utworzyć. Pracownik ma możliwość wyświetlenia wszystkich ofert pracy znajdujących się w systemie.
- Każde ogłoszenie o pracę zawiera planowaną datę zakończenia (**plannedFinish**). System podczas każdego uruchomienia sprawdza aktualność tej daty. W momencie jej osiągnięcia status ogłoszenia zostaje zmieniony z **Active** na **Finished**.
- Po upływie roku od zakończenia ogłoszenia (na podstawie pola **endDate** (data zakończenia działania ogłoszenia) dla ogłoszenia o statusie **Finished**) oferta pracy uzyskuje status **Archived** oraz zostaje zapisana data archiwizacji **archiveDate**.
- Pracownik firmy otrzymuje zgłoszenia kandydatów jako odpowiedzi na oferty pracy drogą mailową. Następnie ręcznie wprowadza dane kandydatów do systemu jeśli jeszcze nie widnieją oni w bazie oraz nowy adres. Kandydat nie ma możliwości zmiany adresu w czasie.
- W przypadku gdy kandydat znajduje się już w systemie, pracownik dodaje tylko nową aplikację powiązaną z odpowiednią ofertą pracy. Kandydat zobowiązany jest również przesłać swoje CV, z którego pracownik ręcznie przepisuje najważniejsze informacje do systemu.
- Jeśli firma „PreczBezrobocie” zatrudnia nowego pracownika, osoba posiadająca dostęp do systemu wprowadza jego dane do bazy.
- W sytuacji gdy firma „PreczBezrobocie” prowadzi rekrutację na własne potrzeby, kandydaci są zapisywane w tabeli **OurCompanyCandidate**. Kandydaci ci muszą dostarczenia listu motywacyjnego (**coverLetter**).

Proces obsługi aplikacji przebiega według następującego scenariusza:

1. Pracownik odbiera zgłoszenie od kandydata.
2. Jeśli kandydat nie istnieje w bazie danych, pracownik dodaje jego dane oraz adres.
3. Pracownik wprowadza dane aplikacji oraz najważniejsze informacje zawarte w CV. Klucz **cvNumber** tworzy się automatycznie.
4. Aplikacja kandydata zostaje zapisana w systemie.
5. Aplikacja zostaje przekazana do firmy ogłaszającej w momencie gdy oferta pracy uzyska status **Finished**.

6. Firma ogłaszająca ocenia aplikację kandydata, nadając jej status swój status według ich systemu. W precz bezrobocie status aplikacji pozostaje jako **Finished**.
7. Aplikacja oraz oferta pozostają w systemie. Oferta po upływie 365 dni od daty **endDate** uzyskuje status **Archived**.

Diagram przypadków użycia



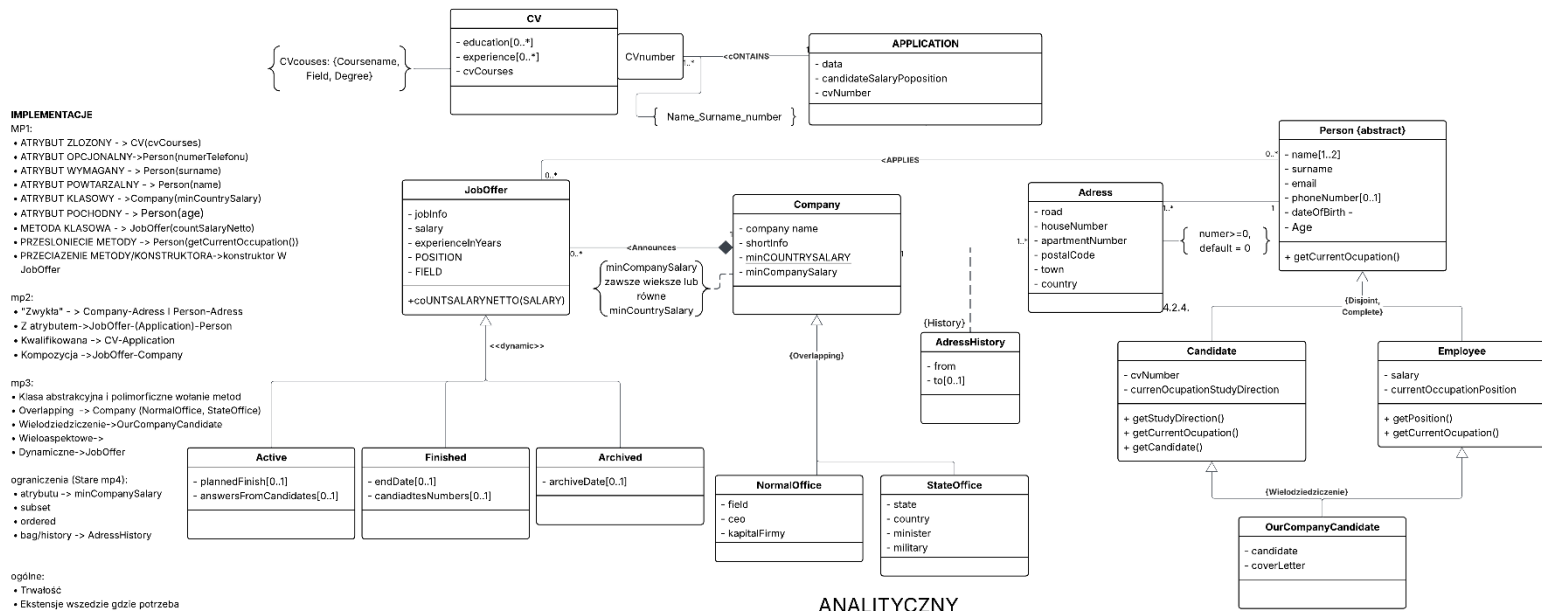
Wymagania niefunkcjonalne systemu

1. System powinien działać jako aplikacja desktopowa uruchamiana lokalnie na komputerze firmy „PreczBezrobocie”.
2. System powinien przechowywać dane w trwałej bazie danych umożliwiające zachowanie informacji po zamknięciu aplikacji.
3. System powinien być uruchamiany przynajmniej raz dziennie.
4. System powinien automatycznie sprawdzać status ofert pracy i ich endDate podczas każdego uruchomienia aplikacji.
5. Dane kandydatów, firm oraz aplikacji powinny być przechowywane w sposób zapewniający spójność danych i integralność relacji między encjami.
6. Usunięcie firmy z systemu powinno powodować automatyczne usunięcie wszystkich powiązanych ofert pracy.
7. Interfejs powinien być prosty i intuicyjny, tak aby pracownik firmy mógł obsługiwać system.
8. System powinien umożliwiać łatwe wyszukiwanie ofert pracy, kandydatów oraz firm znajdujących się w bazie danych.
9. System powinien być odporny na wprowadzanie niepoprawnych danych, np. pustych pól wymaganych lub nieprawidłowych dat.
10. System powinien zapewniać możliwość przechowywania historii adresów firm.
11. Aplikacja powinna być zbudowana w sposób umożliwiający dalszą rozbudowę systemu o nowe funkcjonalności.

12. Dane powinny być dostępne wyłącznie z poziomu komputera firmowego posiadającego aplikację.

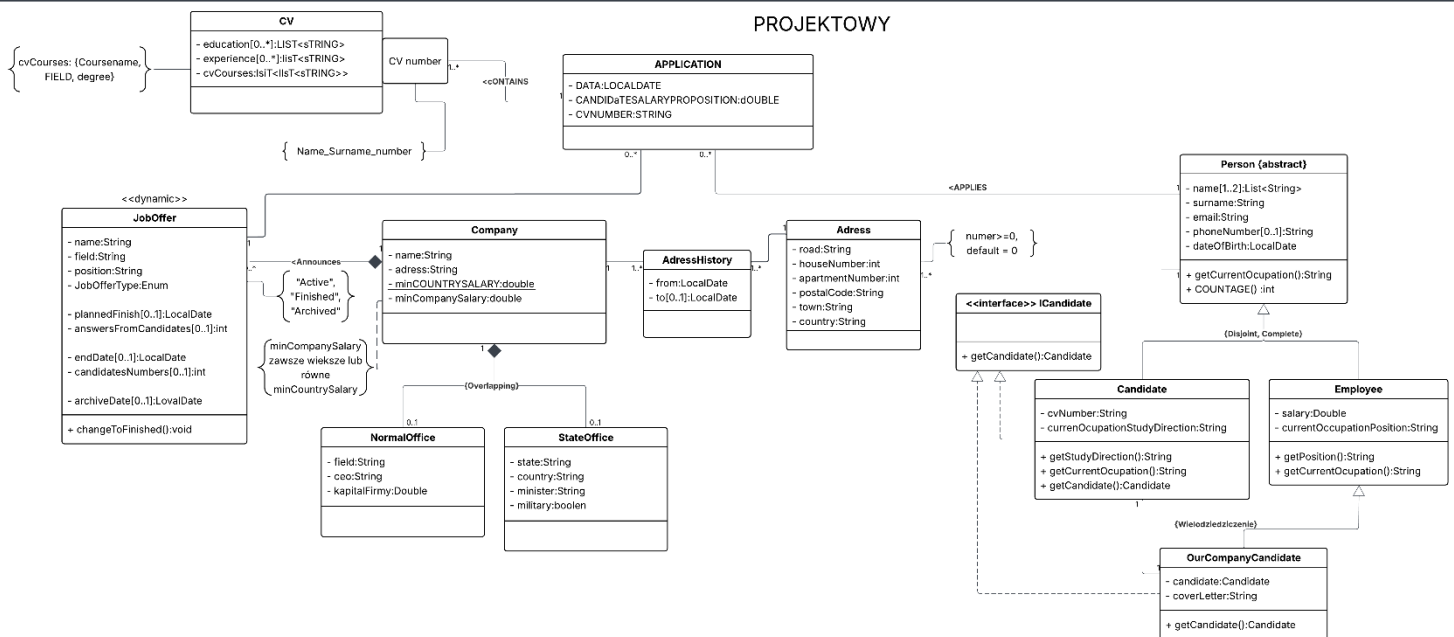
13. System powinien obsługiwać automatyczne zmiany statusów ofert pracy bez konieczności ingerencji użytkownika.

Diagramy klas, analityczny i projektowy



ANALITYCZNY

PROJEKTOWY



Scenariusz przypadków użycia dla przypadku Dodawanie nowej aplikacji kandydata

Aktor: Pracownik agencji z dostępem do systemu

Warunek początkowy: Istnieje co najmniej jedna nowa nie dodana do systemu aplikacja kandydata i istnieje co najmniej jedna firma z ofertą o statusie **Active**

Główny przepływ zdarzeń:

1. Aktor odbiera wiadomość mailową zawierającą zgłoszenie kandydata.
2. Aktor wyszukuje kandydata w bazie danych.
3. System wyświetla dane kandydatów.
4. Aktor wybiera ofertę pracy, której dotyczy zgłoszenie.
5. System wyświetla formularz nowej aplikacji.
6. Aktor wprowadza dane aplikacji.
7. System wyświetla formularz uzupełnienia danych z CV.
8. Aktor wprowadza najważniejsze informacje zawarte w CV.
9. System wyświetla podsumowanie wprowadzonych danych.
10. Aktor potwierdza zapis aplikacji.
11. System zapisuje informację w bazie danych.
12. System wyświetla informację o poprawnym zapisaniu aplikacji.

Alternatywny przepływ zdarzeń:

2a. Kandydat nie istnieje w bazie danych:

1. Pracownik wprowadza dane nowego kandydata.
2. Pracownik dodaje adres nowego kandydata.
3. System zapisuje dane kandydata w bazie danych.
4. System wraca do punktu 3 głównego przepływu zdarzeń.

Warunek końcowy:

Po spełnieniu scenariusza aplikacja kandydata zostaje zapisana w systemie i powiązana z odpowiednią ofertą pracy.

Diagram aktywności dla przypadku użycia

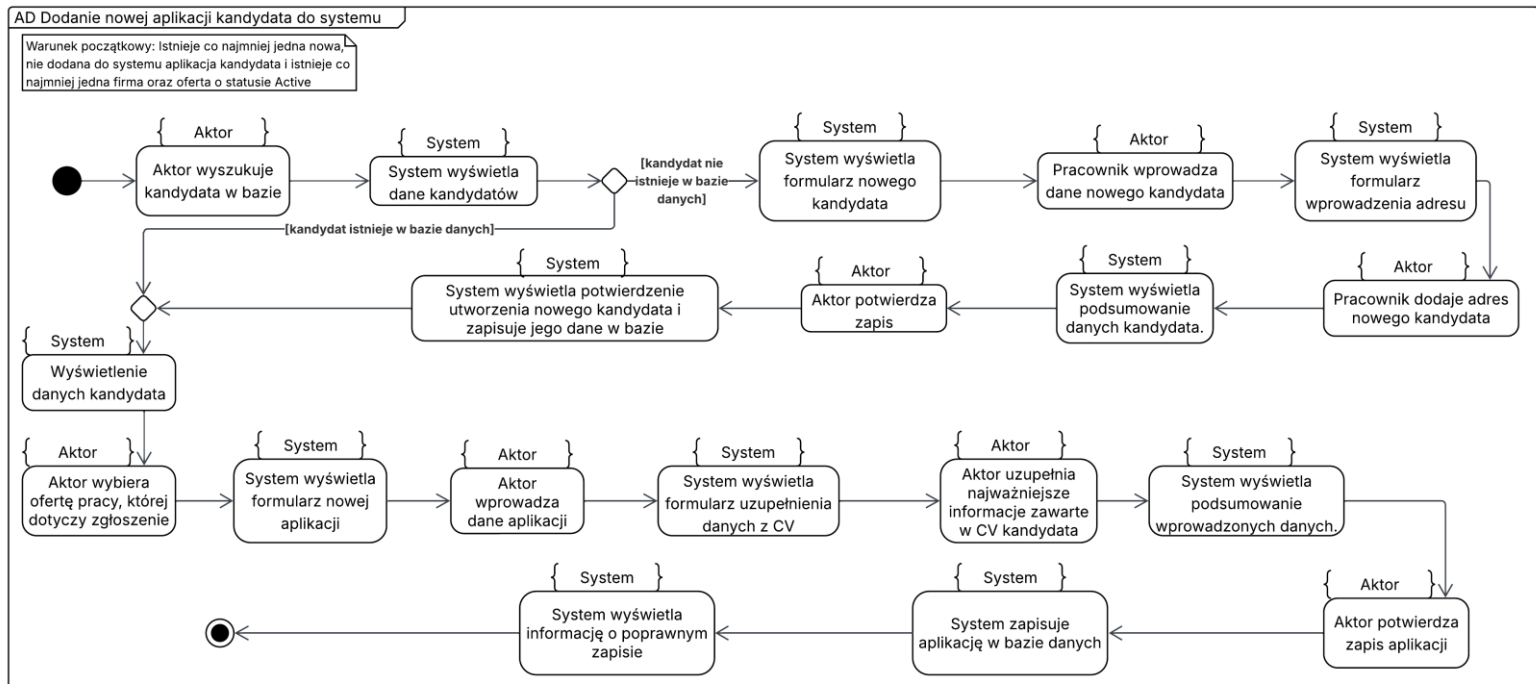
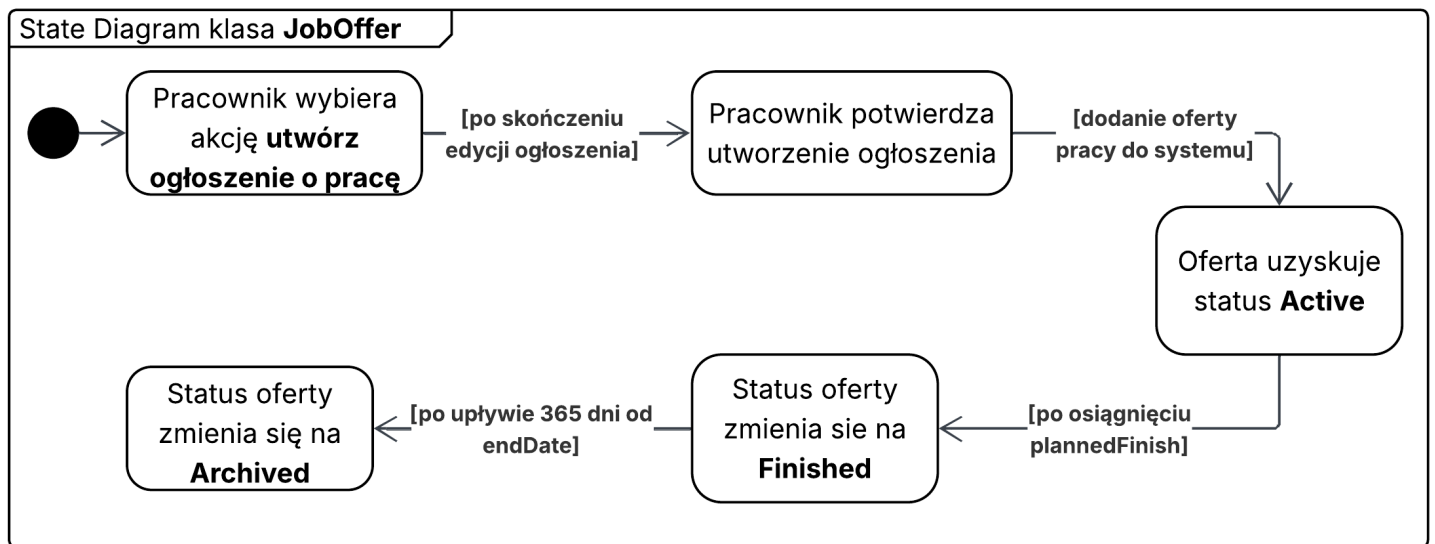
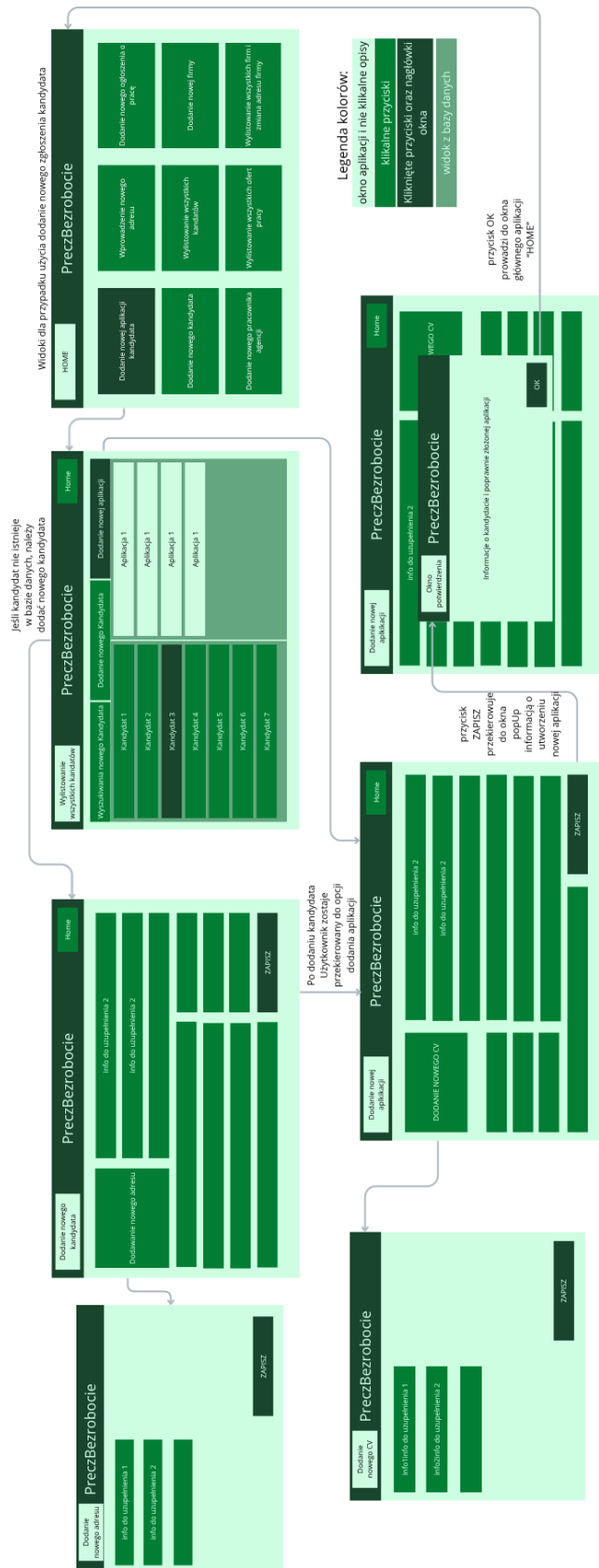


Diagram stanu dla klasy JobOffer



Propozycje GUI



Omówienie decyzji projektowych i skutków analizy dynamicznej.

Technologia i architektura projektu

Wykorzystane technologie:

- Java
 - Główny język implementacji
 - Zapewnia przenośność rozwiązania pomiędzy systemami operacyjnymi
- Hibernate
 - Umożliwia ORM (mapowanie obiektowo-relacyjne)
 - Automatyczne odwzorowanie klas na tabele relacyjnej bazy danych
 - Obsługa relacji, dziedziczenia oraz mechanizmu lazy loading
- Lombok
 - Redukcja kodu szablonowego (getterzy, setterzy, konstruktory)
 - Zwiększenie czytelności klas encyjnych
 - Ograniczenie liczby metod generowanych ręcznie
- PostgreSQL - Baza danych
 - Trwałe przechowywanie danych kandydatów, firm, ofert pracy i aplikacji
 - Komunikacja realizowana poprzez Hibernate
- JavaFX - GUI
 - Warstwa aplikacji łatwa w obsłudze dla użytkownika
 - Nowoczesny interfejs użytkownika
 - Dobra integracja z Javą

Skutki analizy dynamicznej i decyzje projektowe

Podczas analizy zostały przeanalizowane zarówno cykl życia obiektów jak i możliwe zmiany ich stanów w czasie działania systemu. Wnioski z tej analizy wpłynęły na model projektowy:

- Przekształcenie asocjacji z atrybutem **Application**
 - W modelu analitycznym występowała asocjacja **JobOffer** --- **Application** --- **Person**, gdzie **Application** stanowiła atrybut asocjacji pomiędzy ofertami pracy i kandydatami
 - W modelu projektowym **Application** została zamodelowana jako samodzielna klasa encyjna, ponieważ posiada własne dane, oczekiwane wynagrodzenie i numer CV, który jest kluczem obcym do asocjacji kwalifikowanej z klasą **CV**. Klasa **Application** posiada własny cykl życia i powinna być przechowywana w bazie danych.
 - Konsekwencje:
 - Możliwość przechowywania dodatkowych informacji o zgłoszeniu,
 - Możliwość rozbudowy procesu rekrutacyjnego bez modyfikacji relacji między klasami.
- Spłaszczenie dziedziczenia dynamicznego **JobOffer**
 - W modelu analitycznym dziedziczenie pod klasą **JobOffer** reprezentowało stany w których oferta mogła się znajdować, czyli:
 - **Active**,
 - **Finished**,
 - **Archived**.
 - W modelu projektowym dziedziczenie zostało zamienione na atrybuty stanu. Mogło to być wykonane, ponieważ oferta nie zmieniała swojej tożsamości a zmieniał się

tylko jej stan biznesowy i niektóre dane. Pomiedzy stanami występują płynne przejścia. Podklasy nie wprowadzały własnych zachowań ani relacji.

- Konsekwencje:
 - Uproszczenie modelu
 - Mniej tabel w bazie danych
 - Łatwiejsze zarządzanie stanami
- Dziedziczenie overlapping dla **Company**
 - W modelu analitycznym firma mogła należeć do kategorii:
 - **NormalOffice**
 - **StateOffice**

Relacja została oznaczona jako overlapping, ponieważ w momencie, kiedy firma podpisuje kontrakt z organami administracji państwowej lub innych instytucji publicznych, zyskuje prawa i obowiązki instytucji publicznych. Oznacza to możliwość jednoczesnego występowania cech obu tych organizacji.
 - W modelu projektowym pozostawiono możliwość występowania obu ról jednocześnie, zostanie to zaimplementowane za pomocą kompozycji.
 - Konsekwencje:
 - Zachowanie zgodności z rzeczywistością biznesową
 - Brak sztucznego ograniczenia firmy do jednej kategorii
- Historia adresów przedsiębiorstwa
 - Zarówno w modelu analitycznym jak i projektowym zachowano możliwość zapamiętywania informacji historycznych, jakimi jest adres firmy. Zostało to zachowane poprzez stworzenie klasy **AddressHistory**, która przechowuje datę rozpoczęcia i zakończenia obowiązywania adresu.
 - Konsekwencje:
 - Możliwość audytu danych firmy
 - Zachowanie pełnej historii zmian
 - Zgodność z zasadą trwałości danych historycznych
- Wielodziedziczenie w języku Java
 - W modelu analitycznym występuje wielodziedziczenie w klasie **OurCompanyCandidate**, która dziedziczy jednocześnie po klasie **Candidate** i **Employee**.
 - W modelu projektowym wielodziedziczenie zostało zaprezentowane przy użyciu interface'u, ze względu na ograniczenie języka Java, jakim jest brak możliwości dziedziczenia po więcej niż jednej klasie.

Relacja **Employee**->**OurCompanyCandidate** została zamodelowana poprzez zwykłe dziedziczenie, relacja **Candidate**->**OurCompanyCandidate** zostanie zrealizowana poprzez użycie interface'u **ICandidate** oraz smart object copying.

Pozwoli to zachować zarówno cechy pracownika jak i kandydata bez naruszenia ograniczeń języka Java.
 - Konsekwencje:
 - Zgodność z językiem Java
 - Możliwość dalszej rozbudowy modelu
 - Łatwiejsze mapowanie ORM
- Wprowadzenie klasy abstrakcyjnej **Person**
 - W modelu analitycznym i projektowym została zamodelowana klasa **Person**, która została oznaczona jako klasa abstrakcyjna. Zastosowano ją, ponieważ klasy **Candidate** oraz **Employee** współdzielą:

- **Name**
- **Surname**
- **Email**
- **phoneNumber**
- **DateOfBirth**

Klasa ta została stworzona aby uniknąć redundancji danych.

- Konsekwencje:
 - Zgodność z zasadą DRY
 - Centralizacja wspólnych atrybutów
 - Łatwiejsza rozbudowa systemu
- Zastosowanie kompozycji przy asocjacji **Company---JobOffer**
 - Oferta pracy nie może istnieć bez firmy, usunięcie firmy powinno spowodować usunięcie wszystkich jej ofert, a to gwarantuje kompozycja.
 - Konsekwencje:
 - Silna zależność cyklu życia obiektów
- Asocjacja kwalifikowana **CV**
 - Na diagramach asocjacja kwalifikowana występuje dzięki istnieniu kwalifikatora: **cvNumber**. Jest on potrzebny, ponieważ tylko specyficzny (format **Nazwisko_Imie_numer**) indeks jednoznacznie identyfikuje dokument kandydata. Pozwala to na szybkie wyszukiwanie i powiązanie aplikacji z konkretnym CV.
 - Konsekwencje:
 - Prostsze indeksowanie danych
 - Możliwość wykorzystania klucza biznesowego

Potencjalne rozwinięcia systemu

Aplikacja została zaprojektowana tak, aby umożliwić zarówno skalowalność jak i rozwój.

- Zastosowanie Application jako samodzielna encja umożliwia stworzenie rozszerzenia w przyszłości procesu rekrutacyjnego o dalsze etapy, np. rozmowy kwalifikacyjne, czy oceny kandydatów.
- Wydzielenie klasy Adress umożliwia rozwój funkcjonalności związanych z audytem danych i raportowaniem zmian oraz prowadzeniem statystyk.
- Architektura oparta na Hibernate oraz PostgreSQL pozwala na łatwe dodawanie nowych encji i relacji biznesowych
- Użycie JavyFX pozwala na rozbudowę interface'u użytkownika o zaawansowane mechanizmy wyszukiwania i filtrowania danych.
- W przyszłości system może zostać rozszerzony o moduł wysyłania kandydatur bezpośrednio do systemu, bez potrzeby wprowadzania danych ręcznie przez pracownika.
- System może również zostać rozszerzony do samodzielnego portalu pracy, w którym zarówno firmy jak i kandydaci mogliby bezpośrednio wysyłać informacje, a wszystko byłoby zapisywane w pełni automatycznie.
- Możliwe będzie też tworzenie statystyk i analiz rynku pracy bez konieczności przebudowy podstawowego modelu działania systemu